

DySL-VLA: Efficient Vision-Language-Action Model Inference via Dynamic-Static Layer-Skipping for Robot Manipulation

Zebin Yang^{1,2}, Yijiahao Qi⁴, Tong Xie^{1,2}, Bo Yu^{3*}, Shaoshan Liu³, Meng Li^{1,2,5*}

¹Institute for Artificial Intelligence, ²School of Integrated Circuits, Peking University, Beijing, China

³Shenzhen Institute of Artificial Intelligence and Robotics for Society, Shenzhen, China

⁴School of Electronics Engineering and Computer Science, Peking University, Beijing, China

⁵Beijing Advanced Innovation Center for Integrated Circuits, Beijing, China

Abstract

Vision-Language-Action (VLA) models have shown remarkable success in robotic tasks like manipulation by fusing a language model’s reasoning with a vision model’s 3D understanding. However, their high computational cost remains a major obstacle for real-world applications that require real-time performance. We observe that the actions within a task have varying levels of importance: critical steps demand high precision, while less important ones can tolerate more variance. Leveraging this insight, we propose DySL-VLA, a novel framework that addresses computational cost by dynamically skipping VLA layers based on each action’s importance. DySL-VLA categorizes its layers into two types: informative layers, which are consistently executed, and incremental layers, which can be selectively skipped. To intelligently skip layers without sacrificing accuracy, we invent a prior-post skipping guidance mechanism to determine when to initiate layer-skipping. We also propose a skip-aware two-stage knowledge distillation algorithm to efficiently train a standard VLA into a DySL-VLA. Our experiments indicate that DySL-VLA achieves 2.1% improvement in success length over Deer-VLA on the Calvin dataset, while simultaneously reducing trainable parameters by a factor of 85.7 and providing a 3.75× speedup relative to the RoboFlamingo baseline at iso-accuracy. Our code is available on https://github.com/PKU-SEC-Lab/DYSL_VLA.

Keywords

Vision-Language-Action Model, Layer Skipping, Robot Manipulation

1 Introduction

Inspired by the success of Vision-Language Models (VLMs) [1, 21], Vision-Language-Action (VLA) models have emerged [4, 14, 33], enabling a promising paradigm for end-to-end robotic control. By tokenizing robot control signals, these models take images as environmental observations and language instructions as the task goal, then generate the next control command for the robot to fulfill the task [15, 16]. Leveraging the vast, internet-scale knowledge embedded within VLMs [4, 15], VLA models have already shown remarkable generalization capabilities in complex robotic tasks like manipulation [8, 14].

However, deploying VLA models on real-world robots poses a significant challenge. Their immense computational demands lead to high latency and power consumption [17, 32, 38, 42, 44], which conflict with the limited resources and battery capacity of most robotic platforms [13, 30]. Consequently, existing VLA systems,

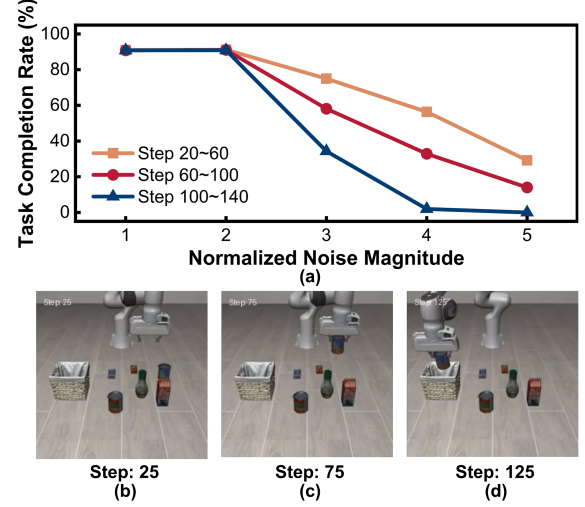


Figure 1: Different actions in robot manipulation have different importance. We show an example when the robot is performing task “Grasp the black cup and drop it into basket”. (a) shows the task completion rates when adding noise with different magnitudes to VLA model weights at different action steps. When adding noise at important action steps, the task completion rate drops faster as noise magnitude increases. We sample 50 times on each noise magnitude for each step range. We show the robot status at (b) step 25, (c) step 75, and (d) step 125 when using the origin VLA model.

like RT-2 (1-3 Hz) [4] and OpenVLA (3-5 Hz) [15], have slow action generation speeds compared to the high-frequency low-level control required for real-time physical interaction (20-50+ Hz) [14].

While existing VLA acceleration methods, such as quantization [6, 24], pruning [5, 45], and knowledge distillation [6, 44], have not fully solved this problem, they often overlook a crucial insight: in robot manipulation, the importance of different actions is not equal. For instance, the act of grasping or releasing an object is far more critical to a task’s success than the preparatory pre-grasp movements, which is also shown in Figure 1. By applying a uniform approach to all action predictions, these methods miss key opportunities for acceleration on less important actions and, therefore, offer limited speedup. Similarly, early exit methods [28, 42] attempt to take advantage of this feature by dynamically adjusting the computational load, but they risk discarding crucial information by exiting before the final layers are fully processed. This trade-off can compromise the model’s overall accuracy and effectiveness [44].

*Corresponding author. Emails: boyu@cuhk.edu.cn, meng.li@pku.edu.cn

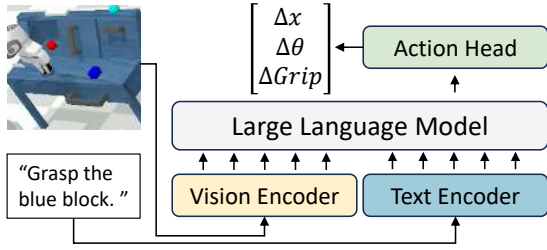


Figure 2: VLA model architecture.

To address the high latency and computational demands of VLA models, we propose DySL-VLA, a method that dynamically skips unnecessary layers during inference. Our approach is based on a key finding: not all VLA layers contribute equally to action prediction. Specifically, we observed that activation distributions change significantly after certain “informative” layers. Our dynamic-static layer skipping method leverages this insight by statically keeping the most critical layers while dynamically skipping others. We also found that the success of a manipulation task is highly sensitive to the accuracy of a few key actions. To account for this and ensure training convergence, we introduce a prior-post skipping guidance and a skip-aware two-stage knowledge distillation method. We summarize our contributions as follows:

- We conduct comprehensive analysis of layer-wise performance and action importance variations in VLA prediction.
- We propose DySL-VLA to accelerate VLA inference via dynamic-static layer skipping. We also propose prior-post skipping guidance and skip-aware two-stage knowledge distillation to ensure the correctness of important actions and improve training convergence.
- Extensive experiments show that DySL-VLA shows 3.75× latency reduction over RoboFlamingo, and 2.1% average successful length improvement over DeeR-VLA, with 85.7× trainable parameters and 13.7× training steps reduction.

2 Background

Vision-language-action Model. Numerous studies have investigated instructing robots by natural language [7, 40, 41]. Among them, VLA models are fine-tuned from pretrained VLMs to increase generalization and conduct robot control in an end-to-end way [2, 43], which shows good performance and becomes the mainstream. As shown in Figure 2, in each frame, VLA model predicts action given current image observation and language instruction [15, 16]. Though achieving high performance, VLA model shows long real-time latency and low control frequency [29, 36], which comes from the high computation cost of the LLM backbone. In this paper, we mainly focus on accelerating the LLM backbone of the VLA models, which accounts for most of the parameters and inference latency (84.3% for OpenVLA [15] and 75.4% for OpenVLA-of [14]).

Efficient Model Inference. Existing VLA acceleration works use pruning [31, 39, 45], quantization [18, 24, 35], and mixture-of-layers [44] to accelerate VLA models, while these methods ignore the importance difference of each action and allocate an equal amount of computation to each prediction, wasting the acceleration

Table 1: Comparison between different methods.

Method	Approach	Action Importance Aware	Specialized Kernel Free	Training Modules
SparseVLM [45]	Pruning	✗	✗	–
FastV [5]	Pruning	✗	✗	–
QAIL [24]	Quantization	✗	✗	LLM Backbone
MoLe-VLA [44]	Mixture-of-Layers	✗	✓	LLM Backbone
DeeR-VLA [42]	Early-exit	✓	✓	LLM Backbone & Action Heads
DySL-VLA	Dynamic-static Layer Skipping	✓	✓	Light-weight Adapters & Skipping Controllers

opportunities on unimportant actions. In addition, methods such as quantization and pruning require specialized kernels, which increase the difficulty of deployment. Early-exit methods [25, 37] halting forward propagation at a certain layer based on intermediate predictions, which can dynamically apply computation on different actions. But skipping all final layers results in a significant loss of information. To solve this, DeeR-VLA [42] largely trains the LLM backbone and multiple action heads to recover model performance. However, this will introduce high computation and memory costs in the training stage. The large-scale fine-tuning on specific scenarios may also break the generalization ability of VLA models.

Compared with existing works, our method adaptively applies more computation to important actions using layer skipping methods. We systematically examine the role of each layer and use dynamic-static layer skipping to reduce information loss after skipping. We use pre-skip prediction and post-skip verification to ensure correct skipping decisions. For training efficiency, we only train light-weight skipping controllers and adapters instead of the LLM backbone. The comparison of different methods for VLA acceleration is shown in Table 1.

3 Efficient VLA Inference via Dynamic-Static Layer-Skipping

3.1 Observation and Overview

To achieve high acceleration rate and model performance after layer skipping, there are two questions to answer. 1) When to skip layers? 2) Which layer to skip?

Observation 1: the importance across different VLA layers varies a lot, while skipping informative layers may cause low model performance. When deciding which layers to skip, existing layer skipping works [9, 20, 26, 42] either empirically skip with the same interval or directly skip all the final layers. However, these strategies do not consider the different importance of VLA layers. We evaluate the amount of information contained in each layer of VLA models by calculating the average cosine similarity of each layer’s output activation. As shown in Figure 3 (a) and (b), we find that some VLA layers significantly change the activation distribution compared to other layers. As shown in Figure 3 (c) and (d), skipping these informative layers will introduce significant performance drops.

Observation 2: VLA systems show high sensitivity on important actions, and more restrictions are needed to decide skipping positions. To decide when to skip layers, existing works [12, 20, 26] use skipping controllers (usually feedforward networks) before LLM layers to predict skipping probability, and conduct layer skipping if the probability exceeds a threshold. However, directly

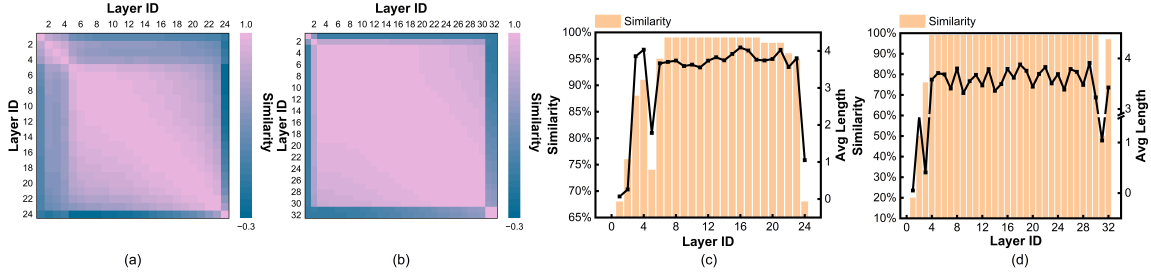


Figure 3: The average cosine similarity between the output activations of different VLA layers for (a) RoboFlemingo-3B and (b) RoboFlemingo-9B. The similarity between the input and output activations of each layer and the model performance when skipping each VLA layer in a zero-shot manner for (c) RoboFlemingo-3B and (d) RoboFlemingo-9B.

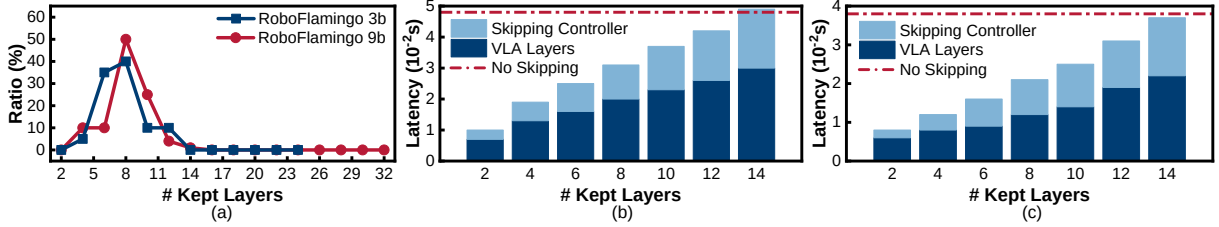


Figure 4: (a) The ratio of different numbers of kept layers in VLA model inference when only using skipping controllers. The inference latency for different numbers of kept layers using (b) RoboFlemingo-3B in FP32 and (c) RoboFlemingo-9B in FP16.

transferring this method to VLA models shows low accuracy. This is because even small errors on important actions may cause task failure, which is shown in Figure 1. However, as shown in Figure 4 (a), when only using skipping controllers, the number of kept layers remains at a relatively low level for all actions. This is because a normalization loss is usually introduced in the training process to force the skipping controllers to skip more layers. And when only using the skipping controllers, the correctness of important actions can not be guaranteed.

In addition, the skipping controllers will introduce non-negligible extra inference latency, which comes from the serial nature of the inference of skipping controllers and the VLA layers [37]. As shown in Figure 4 (b) and (c), when the skipping mechanism is introduced before each layer and half of the layers are activated, the mechanism will gain little latency reduction over the baseline model.

DySL-VLA overview. Based on these observations, we propose DySL-VLA, accelerating VLA models by dynamic-static layer skipping. To achieve low information loss and high speedup, we propose dynamic-static layer skipping to statically keep the informative layers and dynamically skip unnecessary layers (Section 3.2). To keep enough layers for important action, we propose prior-post skipping guidance to guide the skipping decision (Section 3.3). Finally, we propose skip-aware two-stage knowledge distillation to conduct low-cost training and improve training convergence (Section 3.4).

3.2 Dynamic-static Layer Skipping

Problems of existing layer skipping works. Existing layer skipping works do not consider the different importance of VLA layers, thus showing sub-optimal performance. As shown in Figure 5 (b) and (c), early exit methods either need to train multiple action heads [42] or introduce adapters [11] to fit the final action head. However,

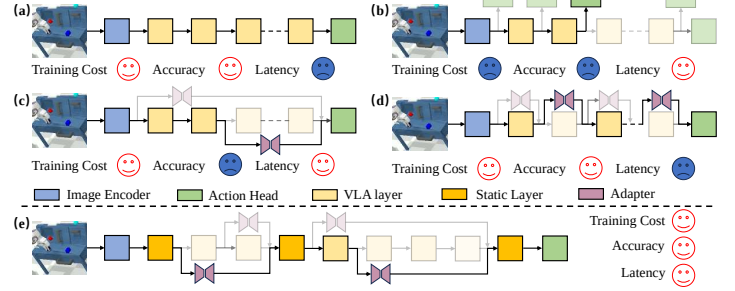


Figure 5: The inference mode of (a) original VLA model, (b) using early exit with multiple action heads, (c) using early exit with adapters, (d) using traditional layer skipping methods, and (e) using dynamic-static layer skipping. The modules with light colour are not activated in current inference. We set the same legend for VLA layers in (a), (b), (c), (d), and dynamic layers in (e).

these methods skip all final layers, some of which are informative, thus showing lower performance. [12, 20] only skip one layer each time, as shown in Figure 5 (d). This fine-grained skipping mechanism shows a low acceleration rate because the skipping controller and adapter introduce extra inference cost.

To solve these problems, we propose a dynamic-static layer skipping mechanism, as shown in Figure 5 (e). As discussed in Section 3.1, some informative layers in VLA models significantly change the activation distribution. We define them as *static layers* and statically keep these layers in model inference. At the same time, although other layers contain less information, we find that directly skipping all of these layers will cause extremely low accuracy. We define these layers as *dynamic layers* and dynamically skip them. Before

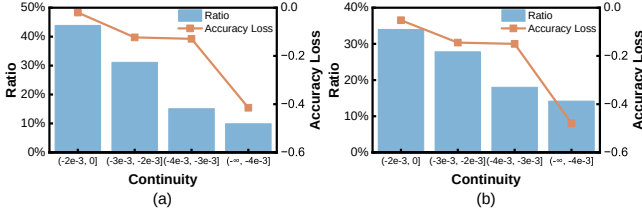


Figure 6: The proportion of action prediction steps in different continuity ranges and the accuracy loss when conducting layer skipping at the steps in different continuity ranges for (a) RoboFlamingo-3B and (b) RoboFlamingo-9B.

each dynamic layer, we determine whether to perform layer skipping (the decision mechanism will be discussed in Section 3.3). If we decide to conduct layer skipping, we directly skip to the next static layer, which shows a higher speedup compared with skipping only one layer each time. And we train adapters (light-weight feedforward layers) to summarize the skipped layers and fit the activation for the next static layer. This is reasonable as the skipped layers will not change the activation distribution much and contain less information, which is within the adapter’s fitting ability.

By using dynamic-static layer skipping, we can achieve a higher speedup with low information loss. At the same time, our method does not need to train multiple action heads or the LLM backbone, which reduces the training cost.

3.3 Prior-post Skipping Guidance

In this section, we discuss our strategy to determine whether to conduct layer skipping before each dynamic layer. As discussed in Section 3.1, just using skipping controllers [12, 20] is neither accurate nor efficient, and we should keep enough layers for important actions. Based on this, we find that the trajectory continuity can reflect the importance of the current action. Here we define the continuity at step t as:

$$C_t = -\frac{1}{k} \sum_{j=t-k+1}^t \|\delta A_j\|_2 = -\frac{1}{k} \sum_{j=t-k+1}^t \|A_j - A_{j-1}\|_2, \quad (1)$$

where δA_j represents the difference between action j and $j-1$, and we consider the trajectory of last k actions. As shown in Figure 6, we find that the trajectory has good continuity at most of the time, which means adjacent actions have similar magnitude and direction. This phenomenon comes from the training data collection process for VLA models, as robot operators tend to keep uniform speeds when executing non-critical motions. However, the continuity will be broken when the robots are conducting fine operations, e.g., grasping or releasing objects. Unlike free-space movements that follow smooth trajectories, these fine operations include frequent stops, micro-corrections, and hesitation, which break the natural flow of motion into disjointed segments [34]. We find that these actions show higher importance in task completion, and conducting layer skipping on these steps will introduce huge accuracy loss, as shown in Figure 6.

Based on these observations, we can approximate action importance based on action continuity to guide layer skipping determination. As shown in Figure 7, besides skipping controllers, we propose

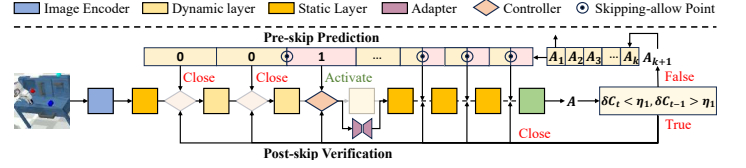


Figure 7: Prior-post skipping guidance. The modules with light colour are not activated in current inference.

pre-skip prediction to approximate the proper skipping positions between static layers. Between each two adjacent static layers, we define a skipping-allow point, which is initialized after the front static layer. Before the point, the skipping controllers are disabled, and the layers are forcibly kept. The skipping controller after the point is activated and determines layer skipping. We dynamically move the skipping-allow point according to the continuity change of the previous k steps:

$$l_i = l_i + \delta l, \text{ when } C_t - C_{t-1} < -\eta \text{ and } l_i < s_i, \quad (2)$$

$$l_i = l_i - 1, \text{ when } C_t - C_{t-1} > \eta \text{ and } l_i > s_{i-1}, \quad (3)$$

where l_i is the id of the layer after the skipping-allow point, η is threshold ($\eta > 0$), s_i is the i_{th} static layer, δl is an adaptive moving stride defined as $\delta l = \lceil \frac{C_t - C_{t-1}}{\eta} \rceil$. Note that when detecting decreased continuity, the skipping allowance point rapidly shifts forward according to continuity change to prioritize action accuracy during critical phases. Conversely, when continuity improves, it gradually moves backward. This hysteresis-like system design maximizes the correctness of essential actions. And we decide the position of the skipping-allow points according to the trajectory of the previous k steps instead of a single recent step ($k = 1$), as the change of action difference ($\delta A_t - \delta A_{t-1}$) will become low when important actions occur in a continuous mode. An example of skipping-allow points moving is shown in Figure 8. Note that the pre-skip prediction only closes or activates the skip controllers, and whether to conduct layer skipping is still determined by the activated skip controller itself.

By using pre-skip prediction, we can keep enough layers for the important actions, thus improving model performance. At the same time, the extra latency cost caused by skipping controllers can also be reduced, as the pre-skip prediction will close most of the skipping controllers that will probably decide not to skip.

However, only using pre-skip prediction is not enough. This is because we can not get the current action prediction before current model inference, so the continuity decrease can only be detected after the first important action has been predicted, whose correctness can not be guaranteed. The issue becomes more pronounced when the action chunk technique is used [2, 14, 36], where multiple actions will be predicted in a single inference. To solve this problem, we also propose post-skip verification to add a feedback mechanism, which is shown in Figure 7. When we first detect the continuity decrease ($\delta C_t = C_t - C_{t-1} < \eta_1$ and $\delta C_{t-1} = C_{t-1} - C_{t-2} > \eta_1$), we re-predict the current action without any layer skipping. Then we recompute the continuity change using the re-predicted action. This process not only ensures the correctness of the initial critical action prediction but also enhances the detection accuracy of continuity degradation. And it will not introduce much extra cost,

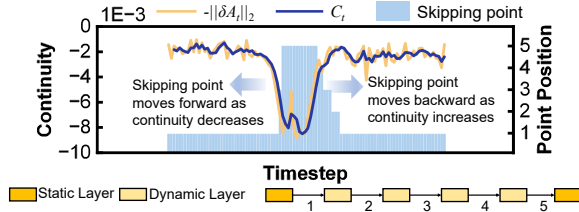


Figure 8: Skipping-allow point changes in different steps. Here we use 4 dynamic layers between 2 static layers as an example, with 5 possible positions for skipping-allow point.

as important actions only occupy a small proportion and usually appear continuously.

3.4 Skip-aware Two-stage Knowledge Distillation

As discussed in Section 2, by freezing the LLM backbone and just training the lightweight skipping controllers and adapters, our training cost can be largely reduced. However, the training strategy is nontrivial, as simply training controllers and adapters together may cause a convergence problem. This is because the controllers and adapters are both randomly initialized, and the training of them will impact each other. At the beginning of training, as the adapters are not trained, the controllers will refuse layer skipping to avoid huge task loss. And this will further affect the adapter training.

To solve this problem, we propose skip-aware two-stage knowledge distillation. In the first stage, as shown in Figure 9 (a), we only train all the adapters to summarize the information of the following dynamic layers. The loss function of the first stage is:

$$loss_1 = \sum_i \|adapter_i(x_i) - L_{s_{i-1}}(L_{s_{i-2}}(\dots(L_i(x_i))))\|_F, \quad (4)$$

where x_i is the input to dynamic layer i , L_i is the i_{th} layer and s_i is the layer id of next static layer.

After the basic capability of adapters are developed, in the second stage, we can then train the controllers and adapters together, as shown in Figure 9 (b). However, in the forward path, if we use the controller itself to decide the skipping place, we find that the model will continuously skip at the first few dynamic layers. So, to fully train each controller, between two static layers, we just select one dynamic layer i ($s_{i-1} < i < s_i$) and predict the probability of skipping using the controller before this layer. In layer selection, it is important to select early dynamic layers more times, as the corresponding adapters are required to summarize more dynamic layers and need more training. In practice, we use Harmonic decay probability [3] to select the dynamic layer, and we find that other strategies, such as linear decay probability, also work well. Different from the inference time, to make the skipping decision module differentiable, we conduct the forward propagation from layer i to layer s_i following:

$$x_{s_i} = controller_i(x_i) \cdot adapter_i(x_i) + (1 - controller_i(x_i)) \cdot L_{s_{i-1}}(L_{s_{i-2}}(\dots(L_i(x_i)))) \quad (5)$$

where x_{s_i} is the activation input to the next static layer. We also introduce a normalization loss to encourage the controller to skip

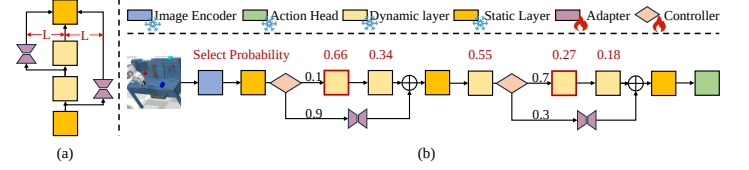


Figure 9: The (a) first stage and (b) second stage of skip-aware two-stage knowledge distillation method. The layers in red boxes are the selected layers in current training step.

layers. The loss function of the second stage is:

$$loss_2 = task_loss + \lambda \cdot \sum_i (1 - controller_i(x_i)) \cdot (s_i - i), \quad (6)$$

where i belongs to the layer ids of the selected layers.

Although our training has two stages, its cost is far lower than previous layer skipping works, as we do not train the LLM backbone and need fewer training steps, which we show in Section 4. Note that our method is totally different from LoRA [10], which uses light-weight adapters to fine-tune the LLM backbone.

4 Experiments

4.1 Experiment Setup

We evaluate DySL-VLA on CALVIN benchmark [23] using RoboFlamingo [16] models and LIBERO benchmark [19] using OpenVLA-ofT models [14]. In the simulation platform, the robot can access RGBD observations. For Calvin dataset, the robot is instructed to complete a task sequence with five subtasks. Following [42], model performance is evaluated based on the average successful length (0 to 5). A larger successful length means more sub-tasks are completed. For OpenVLA-ofT, following [14], we evaluate on 4 sub-datasets. We also deploy our model on the computation platform (Jetson Orin) that is frequently used by real-world robots.

4.2 Main Results

Accuracy comparison. The accuracy comparison is shown in Table 2 and 3. On Calvin D $\rightarrow$ D dataset, compared with traditional methods like HULC and SPIL, DySL-VLA achieves better accuracy and generalization, by leveraging pre-trained VLMs with internet-scale knowledge as backbone. Compared with FlexiDepth [20], which only uses skipping controllers and adapters before each layer to conduct layer skipping, DySL-VLA shows 54.5% average successful length improvement, by using static-dynamic layer skipping to reduce information loss. DySL-VLA also shows 2.1% average successful length improvement over DeeR-VLA, with 85.7 $\times$ trainable parameters reduction and 13.7 $\times$ training steps reduction. Because DySL-VLA keeps the informative layers to avoid information loss, and uses pre-skip prediction and post-skip verification to ensure correction for important actions. On LIBERO dataset, our method also shows 41.3% average SR improvement over FlexiDepth. Compared with DeeR-VLA, our method shows 1.2% average SR improvement, with 31.4 $\times$ trainable parameters reduction.

Latency Comparison. The average LLM latency comparison is shown in Table 2 and 3. On Calvin dataset, our method achieves 3.75 $\times$ latency reduction compared to the full RoboFlamingo model, by dynamically skipping unnecessary VLA layers. Our method

Table 2: Evaluation on Calvin dataset.

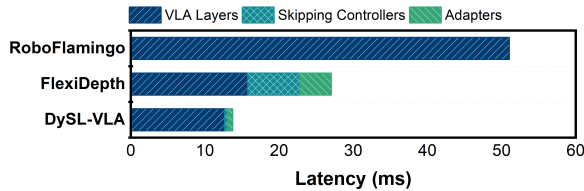
Method	# Fine-tuned Parameters	# Fine-tuned Steps	Training Cost (GPU Hour)	D → D Avg Length	ABC → D Avg Length	RTX 4090 Latency
HULC [22]	–	–	–	2.64	0.67	–
SPIL [46]	–	–	–	2.67	1.71	–
RoboFlamingo 3B [16]	–	–	–	2.92	2.85	51.0ms
Deer-VLA 3B [42]	1.2B	$9.2 \cdot 10^4$	112	2.83	2.82	19.3ms
Random Skip	–	–	–	0.38	0.45	22.6ms
FlexiDepth [20]	19M	$6.7 \cdot 10^3$	7	1.87	1.65	27.6ms
DySL-VLA 3B	14M	$6.7 \cdot 10^3$	7	2.89	2.83	13.6ms

Table 3: Evaluation on LIBERO dataset.

Method	# Fine-tuned Parameters	Spatial SR (%)	Object SR (%)	Goal SR (%)	Long SR (%)	Average SR (%)	A6000 Latency	Jetson Orin Latency
MDT (scratch) [27]	–	78.5	87.5	73.5	64.8	76.1	–	–
π_0 + FAST [2]	–	96.4	96.8	88.6	60.2	85.5	–	–
OpenVLA-OFT 7B [14]	–	97.6	98.4	97.9	94.5	97.1	53.0ms	676ms
Deer-VLA 7B [42]	7.1B	97.0	98.2	97.4	88.6	95.3	40.2ms	495ms
Random Skip	–	11.3	0.6	0.0	0.6	3.1	34.9ms	383ms
FlexiDepth [20]	390M	84.0	85.2	41.4	10.3	55.2	42.2ms	502ms
DySL-VLA 7B	226M	98.0	98.2	97.0	92.6	96.5	27.4ms	345ms

Table 4: Individual influence of our methods (evaluated on Calvin D → D dataset).

Method	Avg Length	Avg Latency
RoboFlamingo 3B [16]	2.82	51.0ms
Random Skip	0.38	22.6ms
DySL-VLA	2.89	13.6ms
w/o Post-skip Verification	2.79	13.4ms
w/o Pre-skip Prediction	2.42	20.1ms
w/o Skip-aware Two-stage Knowledge Distillation	2.82	74.0ms
w/o Dynamic-static Layer Skipping	1.87	27.6ms


Figure 10: Latency breakdown of LLM backbone.

achieves $2.03\times$ latency reduction compared with FlexiDepth [20], which only skips one layer each time, as our method can skip more layers by using static-dynamic layer skipping. And our method avoids redundant skipping controller inference by pre-skip prediction, which is further shown in Figure 10. Our method also shows $1.42\times$ latency reduction compared with early exit method Deer-VLA [42], with far less training cost. Because our method can keep the most informative layers of the original model, thus avoiding huge training costs to recover model performance and achieve high speedup. On LIBERO dataset, our method shows $1.54\times$ and $1.47\times$ latency reduction on A6000 and $1.46\times$ and $1.43\times$ on Jetson Orin, compared with FlexiDepth and Deer-VLA, respectively. Our method shows $1.93\times$ and $1.96\times$ latency reduction on A6000 and Jetson Orin compared with full RoboFlamingo model. This acceleration ratio is lower than RoboFlamingo, as OpenVLA-oft has lower parameter redundancy. Note that although deployment on Jetson Orin shows higher latency because of limited computation

Table 5: Ablation study on static layer ratio, evaluated on Calvin D → D dataset.

Static Layer Ratio	10%	15%	20%	25%	30%
Average Length	2.81	2.84	2.89	2.88	2.89
Average Latency (ms)	12.6	13.1	13.6	14.7	15.8

Table 6: Ablation study on δl (using LIBERO Spatial dataset).

δl	1	2	3	4	5	Adaptive
SR	96.2	96.4	96.4	97.4	97.0	98.0

Table 7: Influence of the value of k in pre-skip prediction, evaluated on Calvin D → D dataset.

k	1	3	5	7	9
Average Length	2.59	2.76	2.89	2.80	2.69

resources compared with A6000, the control frequency of DySL-VLA can still reach 23.2Hz, as OpenVLA-oft uses action chunk and predicts 8 actions in a single inference.

4.3 Ablation Study

Individual influence of our methods. The individual influence of our methods is shown in Table 4. Not using pre-skip prediction and post-skip verification will both cause accuracy drop, as the correctness of important actions can not be fully guaranteed. Without dynamic-static layer skipping, the informative layers can not always be saved, resulting in accuracy drop. Without skip-aware two-stage knowledge distillation, the training of adapters and controllers will affect each other, and the controllers will always be closed, thus leading to higher latency, as discussed in Section 3.4.

Impact of static layer ratio. The impact of static layer ratio is shown in Table 5, evaluated on Calvin D → D dataset. Across various ratios, our method maintains both accuracy and latency within an acceptable range, demonstrating its robustness. Relatively, a lower static layer ratio reduces latency, but causes slight accuracy drop. For a higher ratio, the latency increases with little accuracy gain, as some less informative layers are kept as static layers. So a moderate ratio such as 20% is appropriate.

Impact of skipping-allow point moving stride. In pre-skip prediction, we use an adaptive moving stride based on continuity change when moving the skipping-allow point forward. Here we compare our strategy with constant strides, shown in Table 6. Our method overcomes all constant baselines, as the continuity can better reflect the action importance. We also find that a larger forward-moving stride shows relatively better results, as the hysteresis-like system design can better protect essential actions.

Influence of the Trajectory Length in Pre-skip Prediction In pre-skip prediction, we consider the trajectory continuity of previous k actions. The influence of k value is shown in Table 7. In our experiment, we set $k = 5$, which shows good model performance. And there will be model performance drops when k is too large or too small. When k is too small, the short trajectory can not reflect the action importance well. And a large k will reduce the sensitivity to continuity changes.

5 Conclusion

In this paper, we propose DySL-VLA, accelerating VLA models by dynamically skipping unnecessary layers according to action importance. We propose dynamic-static layer skipping to statically keep the most informative layers and reduce information loss. We propose prior-post skipping guidance to guarantee we keep enough layers for important actions. We propose skip-aware two-stage knowledge distillation to improve training convergence. In experiments, DySL-VLA shows 2.1% average successful length improvement over DeeR-VLA on Calvin D $\rightarrow$ D dataset, with 85.7 $\times$ trainable parameters and 13.7 $\times$ training steps reduction.

6 Acknowledgments

This work was supported in part by New Generation Artificial Intelligence-National Science and Technology Major Project (No. 2025ZD0122502), in part by Beijing Municipal Science and Technology Program under Grant Z241100004224015, in part by NSFC (No. 92464104, 62495102), in part by the National Key Research and Development Program under Grant 2024YFB4505004, and in part by Shenzhen Key Industry R&D Project (No. ZDCY20250901105036006): Research and Development of High-Efficiency Edge Chips for "Brain-Cerebellum" Coordination in Embodied Intelligence.

References

- [1] Jean-Baptiste Alayrac, Jeff Donahue, Pauline Luc, Antoine Miech, Iain Barr, Yana Hasson, Karel Lenc, Arthur Mensch, Katherine Millican, Malcolm Reynolds, et al. 2022. Flamingo: a visual language model for few-shot learning. *Advances in neural information processing systems* 35 (2022), 23716–23736.
- [2] Kevin Black, Noah Brown, Danny Driess, Adnan Esmail, Michael Equi, Chelsea Finn, Niccolo Fusai, Lachy Groom, Karol Hausman, Brian Ichter, et al. 2024. π_0 : A Vision-Language-Action Flow Model for General Robot Control. *arXiv preprint arXiv:2410.24164* (2024).
- [3] Salomon Bochner. 2005. *Harmonic analysis and the theory of probability*. Courier Corporation.
- [4] Anthony Brohan, Noah Brown, Justice Carbajal, Yevgen Chebotar, Xi Chen, Krzysztof Choromanski, Tianli Ding, Danny Driess, Avinava Dubey, Chelsea Finn, et al. 2023. Rt-2: Vision-language-action models transfer web knowledge to robotic control. *arXiv preprint arXiv:2307.15818* (2023).
- [5] Liang Chen, Haozhe Zhao, Tianyu Liu, Shuai Bai, Junyang Lin, Chang Zhou, and Baobao Chang. 2024. An image is worth 1/2 tokens after layer 2: Plug-and-play inference acceleration for large vision-language models. In *European Conference on Computer Vision*. Springer, 19–35.
- [6] Yuxuan Chen and Xiao Li. 2025. RLRC: Reinforcement Learning-based Recovery for Compressed Vision-Language-Action Models. *arXiv preprint arXiv:2506.17639* (2025).
- [7] Danny Driess, Fei Xia, Mehdi SM Sajjadi, Corey Lynch, Aakanksha Chowdhery, Ayaan Wahid, Jonathan Tompson, Quan Vuong, Tianhe Yu, Wenlong Huang, et al. 2023. Palm-e: An embodied multimodal language model. (2023).
- [8] Cunxin Fan, Xiaosong Jia, Yihang Sun, Yixiao Wang, Jianglan Wei, Ziyang Gong, Xiangyu Zhao, Masayoshi Tomizuka, Xue Yang, Junchi Yan, et al. 2025. Interleave-VLA: Enhancing Robot Manipulation with Interleaved Image-Text Instructions. *arXiv preprint arXiv:2505.02152* (2025).
- [9] Siqi Fan, Xin Jiang, Xiang Li, Xuying Meng, Peng Han, Shuo Shang, Aixin Sun, Yequan Wang, and Zhongyuan Wang. 2024. Not all layers of llms are necessary during inference. *arXiv preprint arXiv:2403.02181* (2024).
- [10] Edward J Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, Weizhu Chen, et al. 2022. Lora: Low-rank adaptation of large language models. *ICLR* 1, 2 (2022), 3.
- [11] Yixin Ji, Jikai Wang, Juntao Li, Qiang Chen, Wenliang Chen, and Min Zhang. 2023. Early exit with disentangled representation and equiangular tight frame. In *Findings of the Association for Computational Linguistics: ACL 2023*. 14128–14142.
- [12] Yikun Jiang, Huanyu Wang, Lei Xie, Hanbin Zhao, Hui Qian, John Lui, et al. 2024. D-llm: A token adaptive computing resource allocation strategy for large language models. *Advances in Neural Information Processing Systems* 37 (2024), 1725–1749.
- [13] Leela S Karumbunathan. 2022. Nvidia jetson agx orin series. Online at <https://www.nvidia.com/content/dam/en-zz/Solutions/gtc/t21/jetson-orin/nvidia-jetson-agx-orin-technical-brief.pdf> (2022).
- [14] Moo Jin Kim, Chelsea Finn, and Percy Liang. 2025. Fine-tuning vision-language-action models: Optimizing speed and success. *arXiv preprint arXiv:2502.19645* (2025).
- [15] Moo Jin Kim, Karl Pertsch, Siddharth Karamcheti, Ted Xiao, Ashwin Balakrishna, Suraj Nair, Rafael Rafailov, Ethan Foster, Grace Lam, Pannag Sanketi, et al. 2024. Openvla: An open-source vision-language-action model. *arXiv preprint arXiv:2406.09246* (2024).
- [16] Xinghang Li, Minghuan Liu, Hanbo Zhang, Cunjun Yu, Jie Xu, Hongtao Wu, Chilam Cheang, Ya Jing, Weinan Zhang, Huaping Liu, et al. 2023. Vision-language foundation models as effective robot imitators. *arXiv preprint arXiv:2311.01378* (2023).
- [17] Ye Li, Yuan Meng, Zewen Sun, Kangye Ji, Chen Tang, Jiajun Fan, Xinzhu Ma, Shutao Xia, Zhi Wang, and Wenwu Zhu. 2025. SP-VLA: A Joint Model Scheduling and Token Pruning Approach for VLA Model Acceleration. *arXiv preprint arXiv:2506.12723* (2025).
- [18] Ji Lin, Jiaming Tang, Haotian Tang, Shang Yang, Wei-Ming Chen, Wei-Chen Wang, Guangxuan Xiao, Xingyu Dang, Chuang Gan, and Song Han. 2024. AWQ: Activation-aware Weight Quantization for On-Device LLM Compression and Acceleration. *Proceedings of Machine Learning and Systems* 6 (2024), 87–100.
- [19] Bo Liu, Yifeng Zhu, Chongkai Gao, Yihao Feng, Qiang Liu, Yuke Zhu, and Peter Stone. 2023. Libero: Benchmarking knowledge transfer for lifelong robot learning. *Advances in Neural Information Processing Systems* 36 (2023), 44776–44791.
- [20] Xuan Luo, Weizhi Wang, and Xifeng Yan. 2025. Adaptive Layer-skipping in Pre-trained LLMs. *arXiv preprint arXiv:2503.23798* (2025).
- [21] Yulin Luo, Ruichuan An, Bocheng Zou, Yiming Tang, Jiaming Liu, and Shanghang Zhang. 2024. Llm as dataset analyst: Subpopulation structure discovery with large language model. In *European Conference on Computer Vision*. Springer, 235–252.
- [22] Oier Mees, Lukas Hermann, and Wolfram Burgard. 2022. What matters in language conditioned robotic imitation learning over unstructured data. *IEEE Robotics and Automation Letters* 7, 4 (2022), 11205–11212.
- [23] Oier Mees, Lukas Hermann, Erick Rosete-Beas, and Wolfram Burgard. 2022. Calvin: A benchmark for language-conditioned policy learning for long-horizon

- robot manipulation tasks. *IEEE Robotics and Automation Letters* 7, 3 (2022), 7327–7334.
- [24] Seongmin Park, Hyungmin Kim, Wonseok Jeon, Juyoung Yang, Byeongwook Jeon, Yoonseon Oh, and Jungwook Choi. 2024. Quantization-aware imitation-learning for resource-efficient robotic control. *arXiv preprint arXiv:2412.01034* (2024).
- [25] Haseena Rahmath P, Vishal Srivastava, Kuldeep Chaurasia, Roberto G Pacheco, and Rodrigo S Couto. 2024. Early-exit deep neural network-a comprehensive survey. *Comput. Surveys* 57, 3 (2024), 1–37.
- [26] David Raposo, Sam Ritter, Blake Richards, Timothy Lillicrap, Peter Conway Humphreys, and Adam Santoro. 2024. Mixture-of-depths: Dynamically allocating compute in transformer-based language models. *arXiv preprint arXiv:2404.02258* (2024).
- [27] Moritz Reuss, Ömer Erdiñç Yağmurlu, Fabian Wenzel, and Rudolf Lioutikov. 2024. Multimodal diffusion transformer: Learning versatile behavior from multimodal goals. *arXiv preprint arXiv:2407.05996* (2024).
- [28] Wenxuan Song, Jiayi Chen, Pengxiang Ding, Yuxin Huang, Han Zhao, Donglin Wang, and Haoang Li. 2025. CEED-VLA: Consistency Vision-Language-Action Model with Early-Exit Decoding. *arXiv preprint arXiv:2506.13725* (2025).
- [29] Wenxuan Song, Jiayi Chen, Pengxiang Ding, Han Zhao, Wei Zhao, Zhide Zhong, Zongyuan Ge, Jun Ma, and Haoang Li. 2025. Accelerating Vision-Language-Action Model Integrated with Action Chunking via Parallel Decoding. *arXiv preprint arXiv:2503.02310* (2025).
- [30] Sebastián Valladares, Mayerly Toscano, Rodrigo Tufiño, Paulina Morillo, and Diego Vallejo-Huanga. 2021. Performance evaluation of the Nvidia Jetson Nano through a real-time machine learning application. In *Intelligent Human Systems Integration 2021: Proceedings of the 4th International Conference on Intelligent Human Systems Integration (IHSI 2021): Integrating People and Intelligent Systems, February 22-24, 2021, Palermo, Italy*. Springer, 343–349.
- [31] Hanzhen Wang, Jiaming Xu, Jiayi Pan, Yongkang Zhou, and Guohao Dai. 2025. SpecPrune-VLA: Accelerating Vision-Language-Action Models via Action-Aware Self-Speculative Pruning. *arXiv preprint arXiv:2509.05614* (2025).
- [32] Songsheng Wang, Rucheng Yu, Zhihang Yuan, Chao Yu, Feng Gao, Yu Wang, and Derek F Wong. 2025. Spec-VLA: Speculative Decoding for Vision-Language-Action Models with Relaxed Acceptance. *arXiv preprint arXiv:2507.22424* (2025).
- [33] Yihao Wang, Pengxiang Ding, Lingxiao Li, Can Cui, Zirui Ge, Xinyang Tong, Wenxuan Song, Han Zhao, Wei Zhao, Pengxu Hou, et al. 2025. VLA-Adapter: An Effective Paradigm for Tiny-Scale Vision-Language-Action Model. *arXiv preprint arXiv:2509.09372* (2025).
- [34] Yeping Wang, Carter Sifferman, and Michael Gleicher. 2024. Iklink: End-effector trajectory tracking with minimal reconfigurations. In *2024 IEEE International Conference on Robotics and Automation (ICRA)*. IEEE, 12165–12171.
- [35] Renjie Wei, Songqiang Xu, Linfeng Zhong, Zebin Yang, Qingyu Guo, Yuan Wang, Runsheng Wang, and Meng Li. 2025. LightMamba: Efficient Mamba Acceleration on FPGA with Quantization and Hardware Co-design. *arXiv preprint arXiv:2502.15260* (2025).
- [36] Junjie Wen, Yichen Zhu, Jinming Li, Minjie Zhu, Zhibin Tang, Kun Wu, Zhiyuan Xu, Ning Liu, Ran Cheng, Chaomin Shen, et al. 2025. Tinyvla: Towards fast, data-efficient vision-language-action models for robotic manipulation. *IEEE Robotics and Automation Letters* (2025).
- [37] Jiaming Xu, Jiayi Pan, Yongkang Zhou, Siming Chen, Jinhao Li, Yaoxiu Lian, Junyi Wu, and Guohao Dai. 2025. SpecEE: Accelerating Large Language Model Inference with Speculative Early Exiting. *arXiv preprint arXiv:2504.08850* (2025).
- [38] Siyu Xu, Yunke Wang, Chenghao Xia, Dihao Zhu, Tao Huang, and Chang Xu. 2025. VLA-Cache: Towards Efficient Vision-Language-Action Model via Adaptive Token Caching in Robotic Manipulation. *arXiv preprint arXiv:2502.02175* (2025).
- [39] Yantai Yang, Yuhao Wang, Zichen Wen, Luo Zhongwei, Chang Zou, Zhipeng Zhang, Chuan Wen, and Linfeng Zhang. 2025. EfficientVLA: Training-Free Acceleration and Compression for Vision-Language-Action Models. *arXiv preprint arXiv:2506.10100* (2025).
- [40] Zebin Yang, Tong Xie, Baotong Lu, Shaoshan Liu, Bo Yu, and Meng Li. 2026. KEEP: A KV-Cache-Centric Memory Management System for Efficient Embodied Planning. *arXiv preprint arXiv:2602.23592* (2026).
- [41] Zebin Yang, Sunjian Zheng, Tong Xie, Tianshi Xu, Bo Yu, Fan Wang, Jie Tang, Shaoshan Liu, and Meng Li. 2025. EfficientNav: Towards On-Device Object-Goal Navigation with Navigation Map Caching and Retrieval. *arXiv preprint arXiv:2510.18546* (2025).
- [42] Yang Yue, Yulin Wang, Bingyi Kang, Yizeng Han, Shen zhi Wang, Shiji Song, Jiashi Feng, and Gao Huang. 2024. Deer-vla: Dynamic inference of multimodal large language models for efficient robot execution. *Advances in Neural Information Processing Systems* 37 (2024), 56619–56643.
- [43] Jiazhao Zhang, Kunyu Wang, Rongtao Xu, Gengze Zhou, Yicong Hong, Xiaomeng Fang, Qi Wu, Zhizheng Zhang, and He Wang. 2024. Navid: Video-based vlm plans the next step for vision-and-language navigation. *arXiv preprint arXiv:2402.15852* (2024).
- [44] Rongyu Zhang, Menghang Dong, Yuan Zhang, Liang Heng, Xiaowei Chi, Gaole Dai, Li Du, Dan Wang, Yuan Du, and Shanghang Zhang. 2025. MoLe-VLA: Dynamic Layer-skipping Vision Language Action Model via Mixture-of-Layers for Efficient Robot Manipulation. *arXiv preprint arXiv:2503.20384* (2025).
- [45] Yuan Zhang, Chun-Kai Fan, Junpeng Ma, Wenzhao Zheng, Tao Huang, Kuan Cheng, Denis Gudovskiy, Tomoyuki Okuno, Yohei Nakata, Kurt Keutzer, et al. 2024. Sparsevlm: Visual token sparsification for efficient vision-language model inference. *arXiv preprint arXiv:2410.04417* (2024).
- [46] Hongkuan Zhou, Zhenshan Bing, Xiangtong Yao, Xiaojie Su, Chenguang Yang, Kai Huang, and Alois Knoll. 2024. Language-conditioned imitation learning with base skill priors under unstructured data. *IEEE Robotics and Automation Letters* (2024).